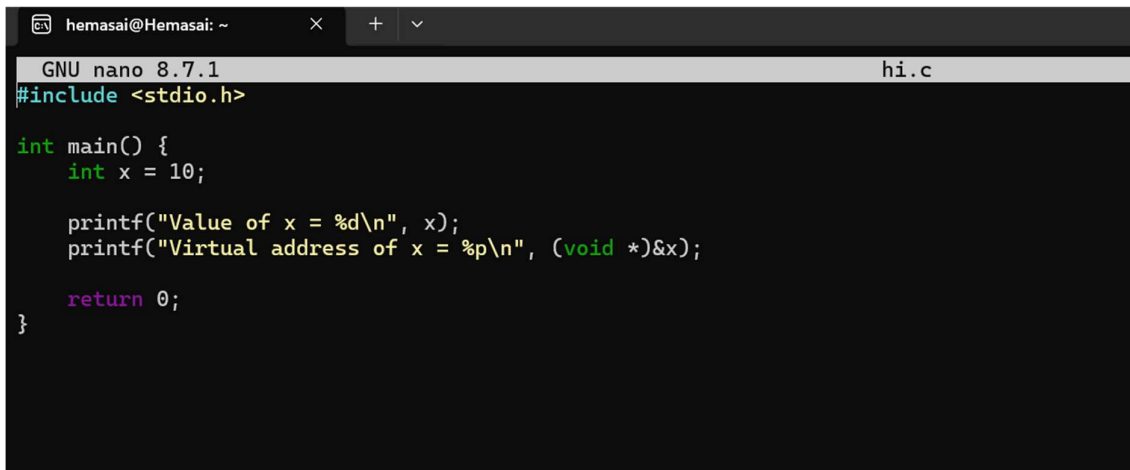


1.virtual address

```
hemasai@Hemasai:~$ nano hi.c
hemasai@Hemasai:~$ gcc hi.c -o hi
hemasai@Hemasai:~$ ./hi
Value of x = 10
Virtual address of x = 0x7ffcd7810744
```



```
hemasai@Hemasai: ~
GNU nano 8.7.1 hi.c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

2.memory layout

```
hi2.c: command not found
hemasai@Hemasai:~$ ./hi2
Code address : 0x600f154581a9
Data address : 0x600f1545b010
BSS address : 0x600f1545b018
Heap address : 0x600f3d8f4010
Stack address : 0x7fff4e72770c
```

```

GNU nano 8.7.1                                h12.c
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address  : %p\n", (void *)main);
    printf("Data address  : %p\n", (void *)&global);
    printf("BSS address   : %p\n", (void *)&bss_variable);
    printf("Heap address   : %p\n", (void *)heap);
    printf("Stack address  : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

3.demand paging

```

GNU nano 8.7.1                                h13.c
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address  : %p\n", (void *)main);
    printf("Data address  : %p\n", (void *)&global);
    printf("BSS address   : %p\n", (void *)&bss_variable);
    printf("Heap address   : %p\n", (void *)heap);
    printf("Stack address  : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

```
hemasai@Hemasai:~$ nano hi3.c
hemasai@Hemasai:~$ gcc hi3.c -o hi3
hemasai@Hemasai:~$ ./hi3
Code address : 0x5dde9e0961a9
Data address : 0x5dde9e099010
BSS address : 0x5dde9e099018
Heap address : 0x5ddeaa4b8010
Stack address : 0x7ffd84791a7c
hemasai@Hemasai:~$ |
```

4.address translation

```
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

```
hemasai@Hemasai:~$ nano hi4.c
hemasai@Hemasai:~$ gcc hi4.c -o hi4
hemasai@Hemasai:~$ ./hi4
Virtual Address = 250
Page Number     = 2
Offset          = 50
Frame Number    = 1
Physical Address = 150
hemasai@Hemasai:~$ |
```

5.memory allocation

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}|
```

```
hemasai@Hemasai:~$ nano hi5.c
hemasai@Hemasai:~$ gcc hi5.c -o hi5
hemasai@Hemasai:~$ ./hi5
10 20 30 40 50
hemasai@Hemasai:~$ |
```

6.segmentation fault

```
GNU nano 2.7.1
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}|
```

```
hemasai@Hemasai:~$ nano hi6.c
hemasai@Hemasai:~$ gcc hi6.c -o hi6
hemasai@Hemasai:~$ ./hi6
Before accessing memory
Segmentation fault ./hi6
hemasai@Hemasai:~$ |
```